

Heap — Complete Interview Guide

1. What is a Heap?

A heap is a **complete binary tree** (array-backed) satisfying the **heap property**:

- **Min-Heap**: $\text{parent} \leq \text{children}$ (root = smallest)
- **Max-Heap**: $\text{parent} \geq \text{children}$ (root = largest)

It is **not** sorted, and it has **no inorder property** (that's a BST thing — see section 9, this is a common point of confusion). A heap only guarantees fast access to the *extreme* element (min or max), not ordering of the rest.

Array representation: for index i , left child = $2i+1$, right child = $2i+2$, parent = $(i-1)/2$.

2. When / Why is it used?

Use a heap whenever you need **repeated access to the min or max element** of a dynamic collection, while elements are being added/removed.

- Sorting gives you full order but costs $O(n \log n)$ upfront and doesn't handle a *streaming* dataset well.
- Array scan for min/max is $O(n)$ every time.
- Heap gives $O(\log n)$ insert and $O(\log n)$ extract-min/max, with $O(1)$ peek. That's the whole reason it exists: **"give me the best/worst element right now, repeatedly, as data keeps changing."**

3. Where is it used (real problems)?

- Top-K elements (largest/smallest/frequent)
- K-way merge (merge K sorted lists)
- Median of a data stream (two heaps)
- Task scheduling / CPU scheduling (priority-based)
- Dijkstra's algorithm, Prim's MST (priority queue)

- Sliding window max/min variants with removal (with lazy deletion)
- Interval scheduling (min-heap of end times)
- Event simulation (min-heap of event times)

4. How do you identify it's a heap problem? (The "2-minute signal")

Ask: **"Do I repeatedly need the min or max of a changing set?"** If yes → heap.

Keyword triggers in the problem statement:

- "Kth largest / Kth smallest / Kth closest"
- "top K frequent / top K elements"
- "merge K sorted lists/arrays"
- "median of a stream"
- "minimum cost to connect / combine" (repeatedly pick 2 smallest)
- "reorganize so no two adjacent are same" (max-heap by frequency)
- "schedule tasks/meetings" with priority or earliest finish time
- "closest points to origin"
- data is described as **streaming / online / arriving one at a time**, and you need a running answer

If the problem needs the *whole sorted order* → just sort, don't use a heap. If it needs *only the extreme value, repeatedly, amid insertions* → heap.

5. Can you identify it from constraints?

Yes — constraints are a strong secondary signal:

Constraint pattern	Suggests
n up to 10^5 – 10^6 , need $O(n \log k)$ not $O(n \log n)$	Heap of size K (Top-K)
K is much smaller than n ($K \ll n$)	Bounded heap of size K
Data arrives incrementally / "design a class with add()"	Heap-based streaming structure

Multiple sorted lists/arrays need merging	K-way merge heap
"minimize/maximize sum after repeatedly combining two elements"	Heap simulation (like Huffman-style)
Need median or running statistic after each insertion	Two-heap pattern

If constraints allow $O(n \log n)$ full sort and you only need order once, a heap is overkill — just sort.

6. Why heap over alternatives — the intuition

- **Heap vs sort:** sort is $O(n \log n)$ once, immutable order. Heap is $O(\log n)$ per operation, built for a *changing* dataset.
- **Heap vs BST (TreeMap/TreeSet):** BST gives full ordering + range queries + $O(\log n)$ arbitrary delete. Heap only gives fast access to *one end* (min or max), but has lower constant factor / simpler implementation. Use BST when you need both ends or ordered iteration; use heap when you only ever care about one extreme.
- **Heap vs QuickSelect:** for a single "find Kth largest" query, QuickSelect averages $O(n)$ (worse case $O(n^2)$ unless randomized/median-of-medians). Heap is $O(n \log k)$, safer worst case, and works well when data is **streaming** (QuickSelect needs the full array upfront).

Core intuition: A heap trades *full order* for *speed at one end*. You pay $O(\log n)$ to maintain just enough structure to always know the extreme value instantly.

7. Types of Heap Patterns

1. **Top-K pattern** — maintain a heap of size K (opposite-type heap trick: min-heap for "K largest", max-heap for "K smallest")
2. **Two Heaps pattern** — max-heap for lower half + min-heap for upper half (running median)
3. **K-way Merge** — min-heap holding one element from each of K lists
4. **Priority Queue / Task Scheduling** — greedy simulation using heap ordering
5. **Interval Scheduling with Heap** — min-heap of end times / meeting rooms
6. **Heap + Greedy combination** — e.g., connect ropes/sticks with min cost, Huffman coding

8. Generic Time & Space Complexity

Operation	Complexity
Build heap from array	$O(n)$
Insert	$O(\log n)$
Extract min/max	$O(\log n)$
Peek	$O(1)$
Top-K over n elements	$O(n \log k)$
K-way merge of n total elements	$O(n \log k)$
Space	$O(n)$ or $O(k)$ for bounded heap

9. Note on “inorder” confusion

Inorder traversal (left → node → right) is a **BST/tree traversal** concept that yields sorted order for a BST — it does **not** apply to heaps, since heaps only guarantee parent-child ordering, not left-right ordering. If a problem mentions “inorder,” it’s signaling a **BST/tree traversal pattern**, not a heap pattern. Don’t conflate the two: heap = “fast access to one extreme,” inorder/BST = “full sorted traversal / range queries.”

10. Reusable Java Templates

Basic Min-Heap / Max-Heap setup

```
// Min-heap (default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max-heap
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

// Custom comparator (e.g., by frequency, by second field, etc.)
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
```

Pattern 1: Top-K Elements

```

public int[] topKLargest(int[] nums, int k) {
    // min-heap of size k -> root is the smallest of the k largest seen so far
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll(); // remove smallest, keep only k largest
        }
    }
    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) {
        result[i] = minHeap.poll();
    }
    return result;
}
// Time: O(n log k), Space: O(k)

```

Pattern 2: Two Heaps (Running Median)

```

class MedianFinder {
    private PriorityQueue<Integer> lowerHalf = new PriorityQueue<>(Collections.re
    private PriorityQueue<Integer> upperHalf = new PriorityQueue<>(); // min-heap

    public void addNum(int num) {
        lowerHalf.offer(num);
        upperHalf.offer(lowerHalf.poll()); // balance step
        if (upperHalf.size() > lowerHalf.size()) {
            lowerHalf.offer(upperHalf.poll());
        }
    }

    public double findMedian() {
        if (lowerHalf.size() > upperHalf.size()) return lowerHalf.peek();
        return (lowerHalf.peek() + upperHalf.peek()) / 2.0;
    }
}
// Time: O(log n) per insert, O(1) per median query

```

Pattern 3: K-Way Merge

```

public int[] mergeKSortedArrays(int[][] arrays) {
    // element = {value, arrayIndex, elementIndex}
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    for (int i = 0; i < arrays.length; i++) {

```

```

        if (arrays[i].length > 0) {
            pq.offer(new int[]{arrays[i][0], i, 0});
        }
    }

    List<Integer> result = new ArrayList<>();
    while (!pq.isEmpty()) {
        int[] top = pq.poll();
        result.add(top[0]);
        int arrIdx = top[1], elIdx = top[2];
        if (elIdx + 1 < arrays[arrIdx].length) {
            pq.offer(new int[]{arrays[arrIdx][elIdx + 1], arrIdx, elIdx + 1});
        }
    }
    return result.stream().mapToInt(Integer::intValue).toArray();
}
// Time: O(n log k) where n = total elements, k = number of arrays

```

Pattern 4: Greedy Simulation (e.g., Connect Ropes / Min Cost)

```

public int minCostToConnectRopes(int[] ropes) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int r : ropes) minHeap.offer(r);

    int totalCost = 0;
    while (minHeap.size() > 1) {
        int first = minHeap.poll();
        int second = minHeap.poll();
        int cost = first + second;
        totalCost += cost;
        minHeap.offer(cost);
    }
    return totalCost;
}
// Time: O(n log n), Space: O(n)

```

Pattern 5: Task/Interval Scheduling (Min Meeting Rooms)

```

public int minMeetingRooms(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by start time
    PriorityQueue<Integer> endTimes = new PriorityQueue<>(); // min-heap of end t

    for (int[] interval : intervals) {
        if (!endTimes.isEmpty() && endTimes.peek() <= interval[0]) {

```

```

        endTimes.poll(); // reuse a room
    }
    endTimes.offer(interval[1]);
}
return endTimes.size();
}
// Time: O(n log n), Space: O(n)

```

Pattern 6: Top-K Frequent Elements

```

public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.merge(n, 1, Integer::sum);

    // min-heap by frequency, size k
    PriorityQueue<int[]> heap = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
        heap.offer(new int[]{e.getKey(), e.getValue()});
        if (heap.size() > k) heap.poll();
    }

    int[] result = new int[k];
    for (int i = k - 1; i >= 0; i--) result[i] = heap.poll()[0];
    return result;
}
// Time: O(n log k), Space: O(n)

```

11. Quick Comparison Table

Pattern	Heap type	When to use	Complexity
Top-K	Bounded opposite heap	"Kth largest/smallest", "top K frequent"	$O(n \log k)$
Two Heaps	Max-heap + min-heap balance	Running median, balance two halves	$O(\log n)/op$
K-way Merge	Min-heap of size k	Merge K sorted lists/arrays	$O(n \log k)$
Greedy Simulation	Min or max-heap	Repeatedly combine 2 smallest/largest	$O(n \log n)$
Interval + Heap	Min-heap of end times	Meeting rooms, resource allocation	$O(n \log n)$

12. Interview Checklist (say this to yourself in <2 min)

1. Do I need the **min or max repeatedly**, not full sorted order? → heap.
2. Is data **streaming/incremental**, or do I need a **running** answer? → heap.
3. Is there a **K** in the problem (Kth, top K, K-way)? → strong heap signal.
4. Do I need **both ends** (median, balance)? → two-heap pattern.
5. Am I told to fully sort or need **ordered range queries**? → not heap, use sort/TreeMap.
6. Constraint says n is huge but K is small, and $O(n \log n)$ full sort feels wasteful? → bounded heap of size K .